Array and Its Processing - Python

WI1102 Computational Thinking Teaching Team

Institut Teknologi Bandung © 2026

Objectives

- Students will be able to:
  - Explain the definition and purpose of arrays
  - Correctly use array definition and referencing notation
  - Perform sequential processing on arrays

Name Combination – 3 Names

- Write a program that accepts three names and then displays all combinations of name pairs.
- Example output:

```
Ali
Budi
Caca
Ali - Budi
Ali - Caca
Budi - Caca
```

```
# DECLARATIONS
# name1, name2, name3 : string

# ALGORITHM
name1 = input()
name2 = input()
name3 = input()

print(name1, " – ", name2)
print(name2, " - ", name3)
print(name3, " – ", name1)
```



Name Combination – 10 Names

- Write a program that accepts 10 names and then displays all combinations of name pairs.
- Example output:

Ali - Budi
Ali - Caca
…
Ina - Jaja

```
# DECLARATIONS
# name1,name2,name3,name4,name5 : string
# name6,name7,name8,name9,name10 : string

# ALGORITHM
name1 = input()
name2 = input()
name3 = input()
# … Continue on your own
name10 = input()

print(name1, " - ", name2)
print(name2, " - ", name3)
print(name3, " - ", name4)
# … Continue on your own
print(name10, " - ", name1)
```



What if…

You are asked to display all possible name-pair combinations from …

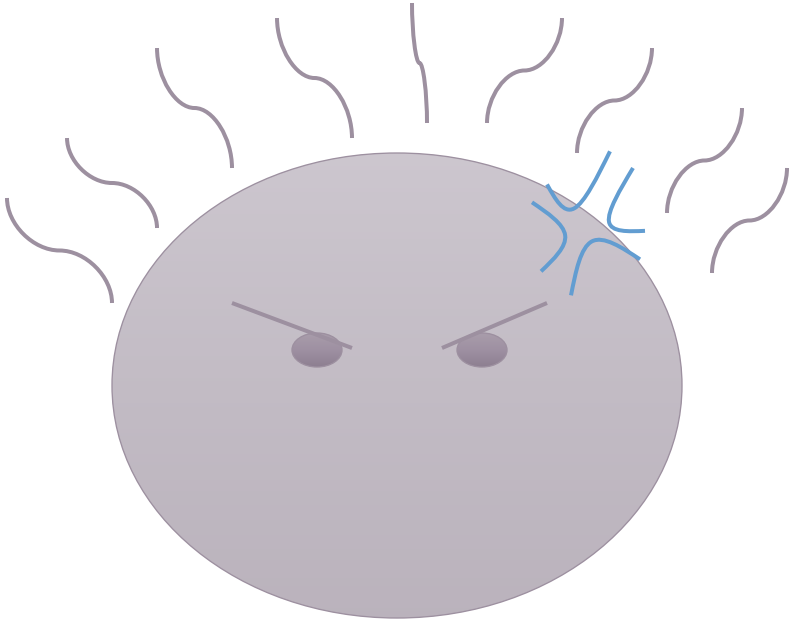
100 names ???

1000 names ???

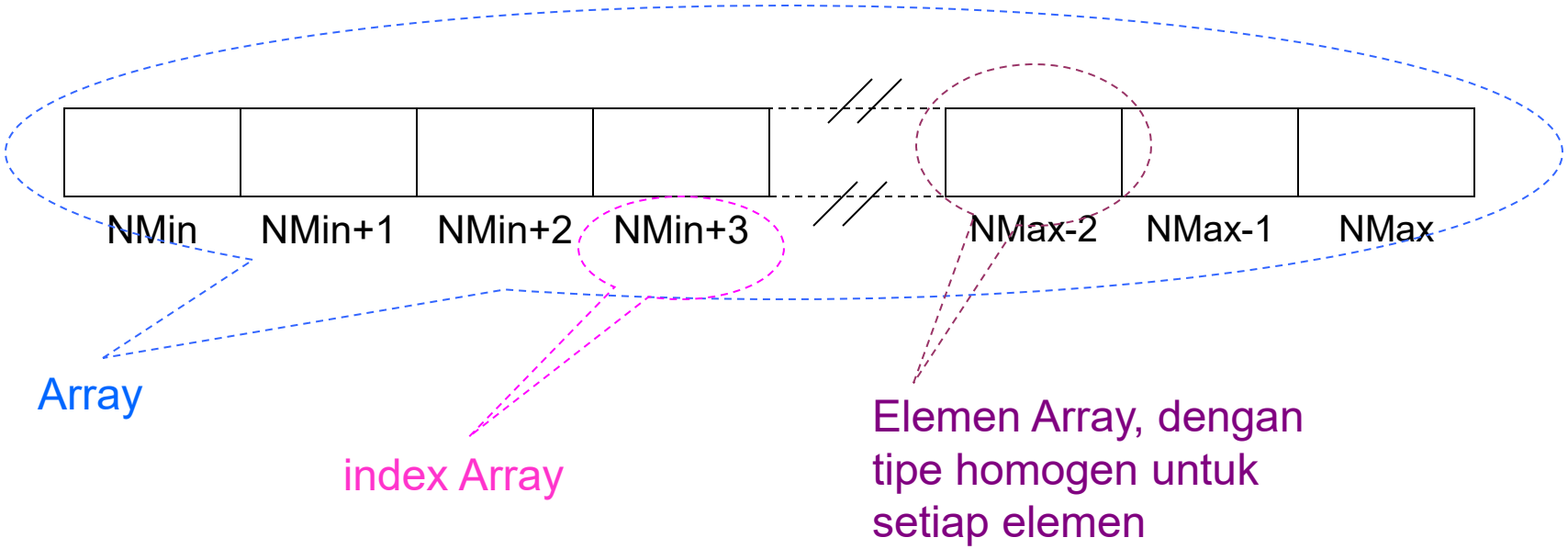
10000 names ???

1000000 names ???

….



Array / Vector / Table



- An **array** defines a **collection** (one or more) of elements **of the same type**.
- Each element is arranged sequentially (contiguously) and can be accessed using an **index**.
- In Python, there are several ways to declare an array; in this lecture, arrays are defined using the `list` collection type.

Array Declaration in Python (1)

Method-1:

- Declaring an array variable while simultaneously defining its contents:

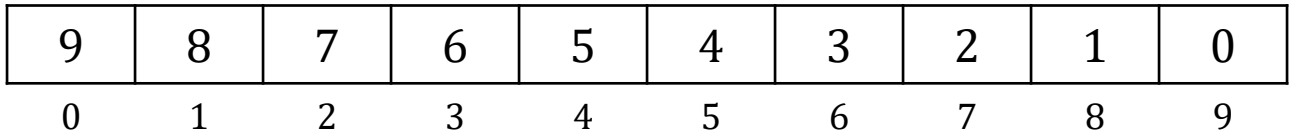
  <name-var> = [<val_0>, <val_1>, <val_2>, … ,< val_{n-1}>]

  - Declare an array named <name-var> of size n with elements <val_0>, <val_1>, <val_2>, … ,< val_{n-1}>
  - The element type depends on the assigned element value.
  - Elements are ordered by index from 0 to n-1.

Array Declaration in Python (2)

- Example-1:

  TabInt = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]



An array named **TabInt** with elements of type **integer** and a size of **10** where the array element addresses (indices) range from index from **index 0 to 9**

Array Declaration in Python (3)

- If the values to be assigned to the array are not yet known, a uniform default value can be assigned initially.
  - Example: An array of integers—default element value = 0.
- **Method-2**: Declaring an array and filling it with default values:

  <name-var> **= [**<default-val> **for i in range (**<n>**)]**

  - Declare an array named <name-var> of size <n> with each element initialized to <default-val>. The variable i is used in the loop to assign the default value to each array element.
  - The element type depends on the value <default-val>
  - Elements are ordered by index from 0 to n-1.

Array Declaration in Python (3)

- Example-2: Array of integer

  TabInt = [0 for i in range(10)]

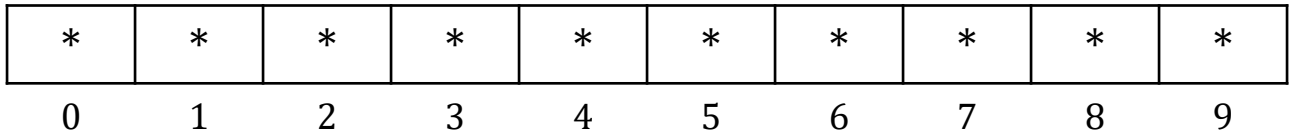
0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

An array named **TabInt** with the size of **10 –** each element is of type **integer** and initialized to the default value of **0** where each element is accessed using indices ranging from **index 0 to 9**

Array Declaration in Python (3)

- Example-3: Array of character

  TabChar = [‘*’ for i in range(10)]



An array named **TabChar** with a size of **10** elements – each element is of type **char** and initialized to the default value *, where each element is accessed using indices ranging from **index 0 to 9**

Accessing Array Elements in Python

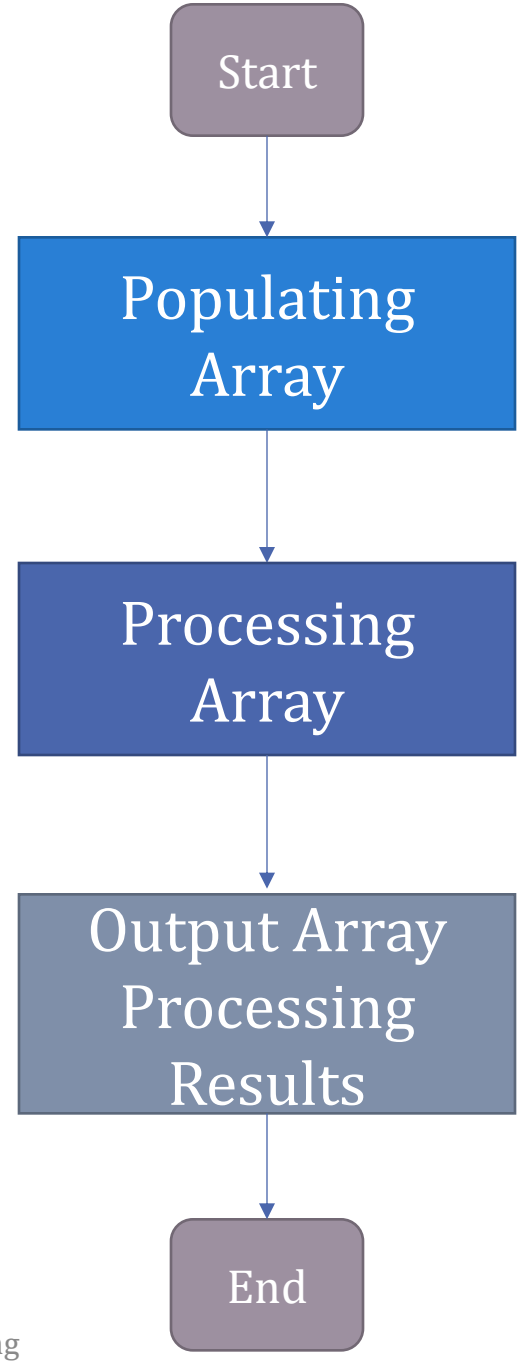
- How to access an element: `<table_name>[<index>]`
- Example: `TabInt = [1,2,4,-1,100,2,0,-1,3,9]`

1	2	4	-1	100	2	0	-1	3	9
0	1	2	3	4	5	6	7	8	9

```
print(TabInt[5])              # prints: 2
x = TabInt[1] + TabInt[6]     # x = 2 + 0 = 2
TabInt[9] = 9                 # Element of index 9 becomes 9
TabInt[10] ???                # Out of range, undefined!!
```

- **Note**: Accessing elements with an index **outside the defined range is not permitted.**
  - In the example above, for instance: **TabInt[10], TabInt[-1]**, etc.

Array Processing



Sequential Processing on Arrays (1)

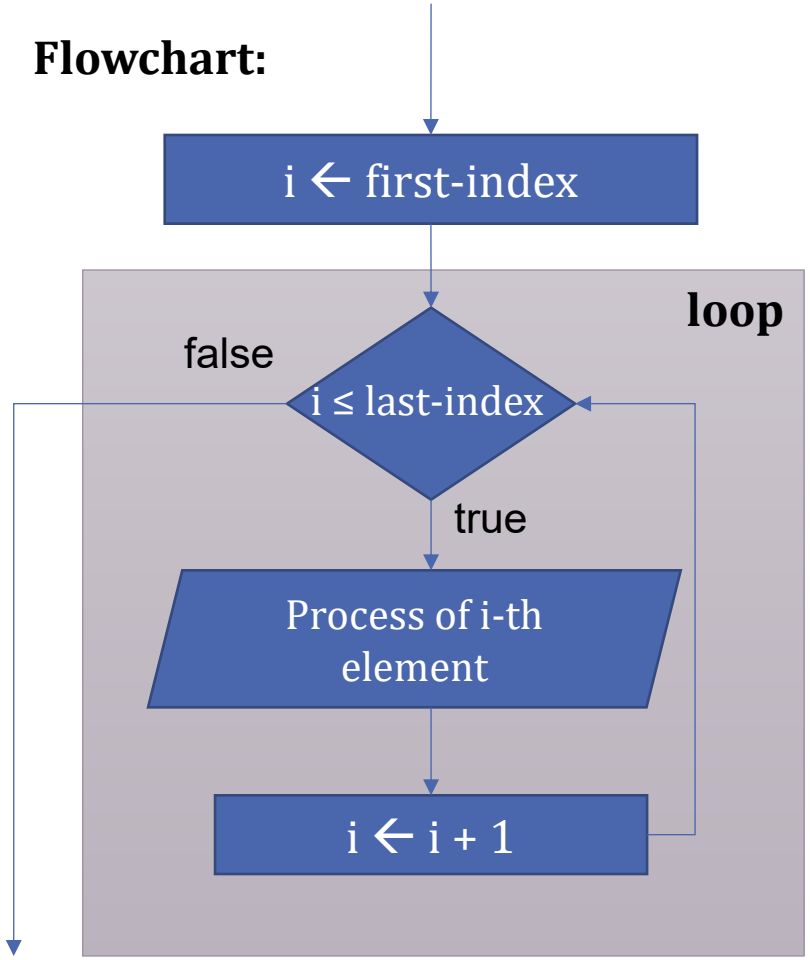
- **Sequential processing** of an array involves processing each array element—starting from the element at the **lowest index** and proceeding to the **highest index** —using a loop.
  - Each array element is accessed directly via its index.
  - The first element is the array element with the lowest index.
  - The next element is reached by moving to the successor index.
  - The termination condition is met when the index being processed is the highest previously defined index.
- **The array is non-empty**, meaning it contains at least one element.

Sequential Processing on Arrays (2)

- Examples of sequential processing tasks on arrays:
  - Populating an array sequentially
  - Printing array elements
  - Calculating the average value of array elements
  - Multiplying array elements by a value
  - Finding the maximum/minimum value in an array
  - Finding the index where a value first appears in an array
  - …

General Flowchart + Pseudocode Sequential Processing on Arrays

Flowchart:



Pseudocode:

```
i traversal [first-index..last-index]
    { Process of i-th element }
    ...
```

Populating An Array (1)

- Write a program that declares an array of 10 integers and populates it with values read from the keyboard.

- Be careful not to access elements outside the array's index bounds!

```
# Program PopulatingArray
# Populating an array with
# values from the user

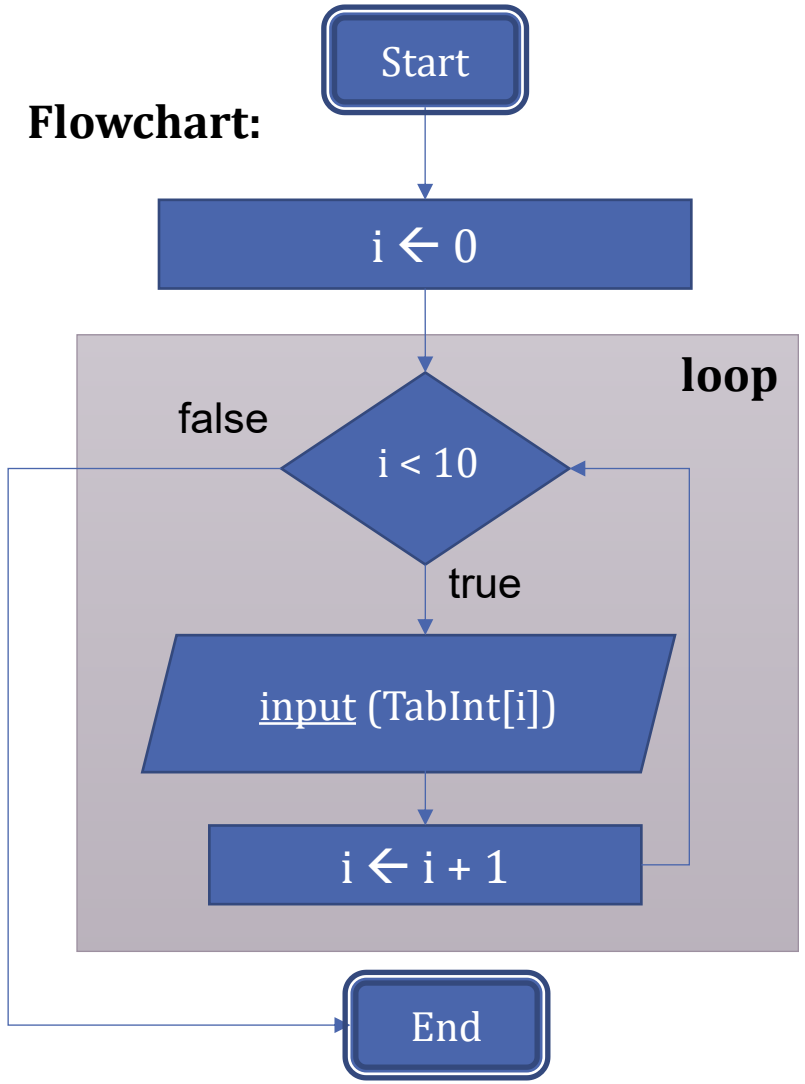
# DECLARATIONS
# TabInt : array [0..9] of int
# i : int

# ALGORITHM
# Declare array TabInt and
# populate it with the default value 0
TabInt = [0 for i in range(10)]

# Populating an array with values read
# from the keyboard
for i in range(0,10):
    TabInt[i] = int(input())
```

Populating An Array (2): Flowchart + Pseudocode

Flowchart:



Pseudocode:

```
i traversal [0..9]
    input(TabInt[i])
```

Displaying the Contents of an Array (1)

- Write a program that:
  - declares an array of 10 integers
  - populates it with values read from the keyboard
  - displays the contents of the array on the screen

- Be careful not to access elements outside the array's index bounds!

```
# Program DisplayArray
# Populates an array with values from#
# the user and displays its contents
# to the screen

# DECLARATIONS
# TabInt : array [0..9] of int
# i : int

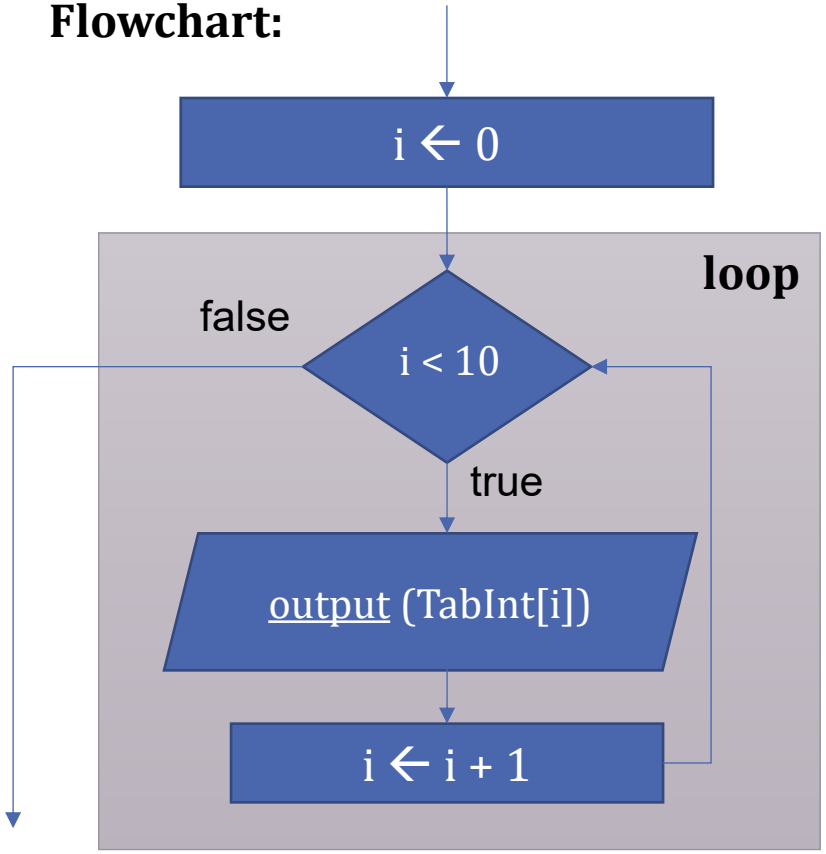
# ALGORITHM
# Declare array TabInt and initialize
# it with a default value of 0
TabInt = [0 for i in range(10)]

# Populate the array by reading
# values from the keyboard
for i in range(0,10):
    TabInt[i] = int(input())

# Print the array contents
for i in range(0,10):
    print(TabInt[i])
```

Displaying the Contents of an Array(2)

Flowchart:



Pseudocode:

```
... { Array populating part }
i traversal [0..9]
    output(TabInt[i])
```

Try writing the array-populating part yourself as an exercise.

Calculating the Average (1)

- Write a program to calculate the average value of the elements in an array.

- Steps:
  - Declare an array (e.g., an array of integers with a size of 10).
  - Populate the array elements.
  - Sum all the array elements.
  - Divide the sum by the number of elements and display the result.

```
# Program AverageArray
# Calculates the average value of array elements

# DECLARATIONS
# TabInt : array [0..9] of int
# i, sum : int
# avg : float

# ALGORITHM
# Declare array TabInt and initialize it with
# a default value of 0
TabInt = [0 for i in range(10)]

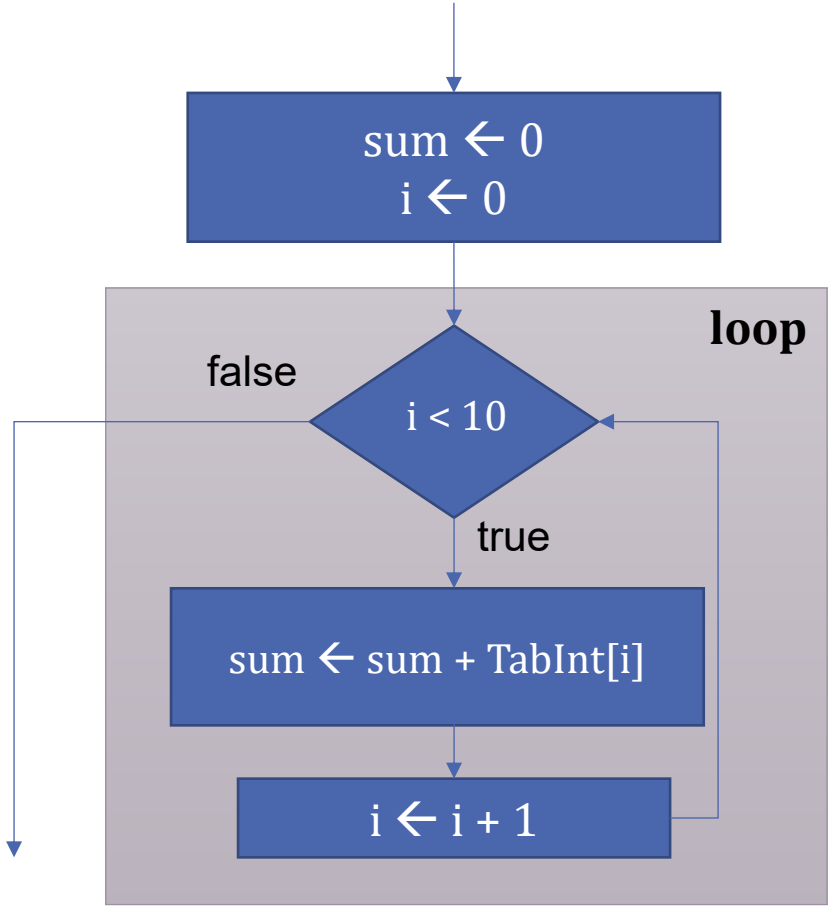
# Populate the array by reading values from the
# keyboard
for i in range(0,10):
    TabInt[i] = int(input())

# Sum the array elements
sum = 0
for i in range(0,10):
    sum = sum + TabInt[i]

# Calculate the average value and display it
avg = sum/10
print ("Nilai rata-rata = " + str(avg))
```

Calculating the Average (1)

Flowchart – Element summation part



Pseudocode – Element summation part

```
... { Array populating part }
sum ← 0
i traversal [0..9]
    sum ← sum + TabInt[i]
...
```

Try to write other parts yourself as an exercise.

Exercises

- For each practice problem, create a flowchart/pseudocode and Python program code.

Exercise-1

- Write a program containing an integer array of size 20, for example, **T**.
- Assume that the part of the program responsible for populating array T already exists.
- Refer to the previous slide regarding array input from the keyboard.
- The program accepts an integer input, for example, X.
- Next, the program multiplies all elements of array T by X and prints the new elements of T to the screen.

Example:

T = [4, 1, 3, 4, 5, 6, 8, 9, 12, 30, -1, 0, 4,
-1, 3, 10, 14, 6, 7, 0]

X = 3

After all elements of T are multiplied by X

T = [12, 3, 9, 12, 15, 18, 24, 27, 36, 90,
-3, 0, 12, -3, 9, 30, 42, 18, 21, 0]

Exercise-2

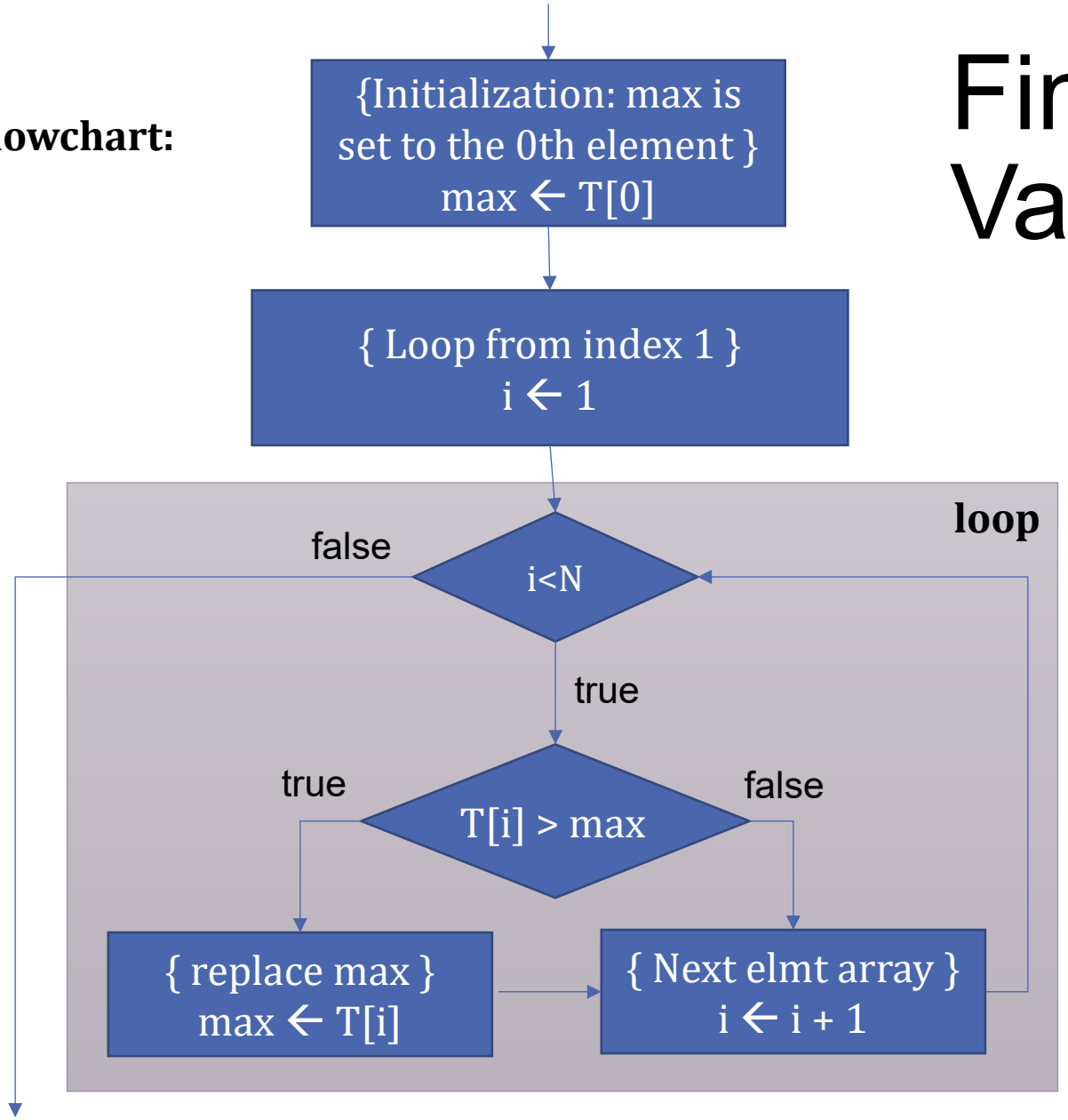
- Student grades are represented by the letters A, B, C, D, and E.
  - Although ITB uses the grades AB and BC, these are disregarded here to simplify the problem.
- A program receives grade data for 50 students in a class—using the letter grades mentioned above—and stores them in an array of characters.
- Determine how many students passed and how many did not. A student is considered to have passed if they received a grade of A, B, or C; otherwise, they did not pass.

Finding Extreme Values (1) (Minimum or Maximum)

- Finding the **largest** or **smallest** value among the elements of an array
- Given:
  - An array T of size N elements
  - Value X (of the same data type as the elements of T)
- Write a program to display the largest value among the elements of T
- Assumption: T is not empty (minimum 1 element, N > 0)
- Examples:
  - N = 10; T contains: [9, 12, 30, -1, 0, 4, -1, 3, 30, 14] → largest value = 30
  - N = 8; T contains: [1, 3, 5, 8, -12, 90, 3, 5] → largest value = 90

Finding Extreme Values (2)

Flowchart:



Pseudocode:

```
...
{ initialize max with 0-th element }
max ← T[0]

i traversal [1..N-1]
   { replace max when T[i]>max }
   if (T[i]>max) then
       max ← T[i]

...
```

```
# Program MaxArray
# Find the largest element of an array

# DECLARATIONS
# N : int
# T : array [0..N-1] of int
# i : int
# max : int

# ALGORITHM
N = 10  # assign N with the size of T
# Assumption: array has been populated
# You still need to write it to test the program

# Finding largest element
max = T[0]   # init max with first elmt.

# Starting from 2-nd element
for i in range(1,N):
    # replace max if curr elmt > max
    if (T[i] > max):
        max = T[i]

# Print the largest element
print ("Largest element = " + str(max))
```

Finding Extreme Values (3) Python Program

Exercise-3

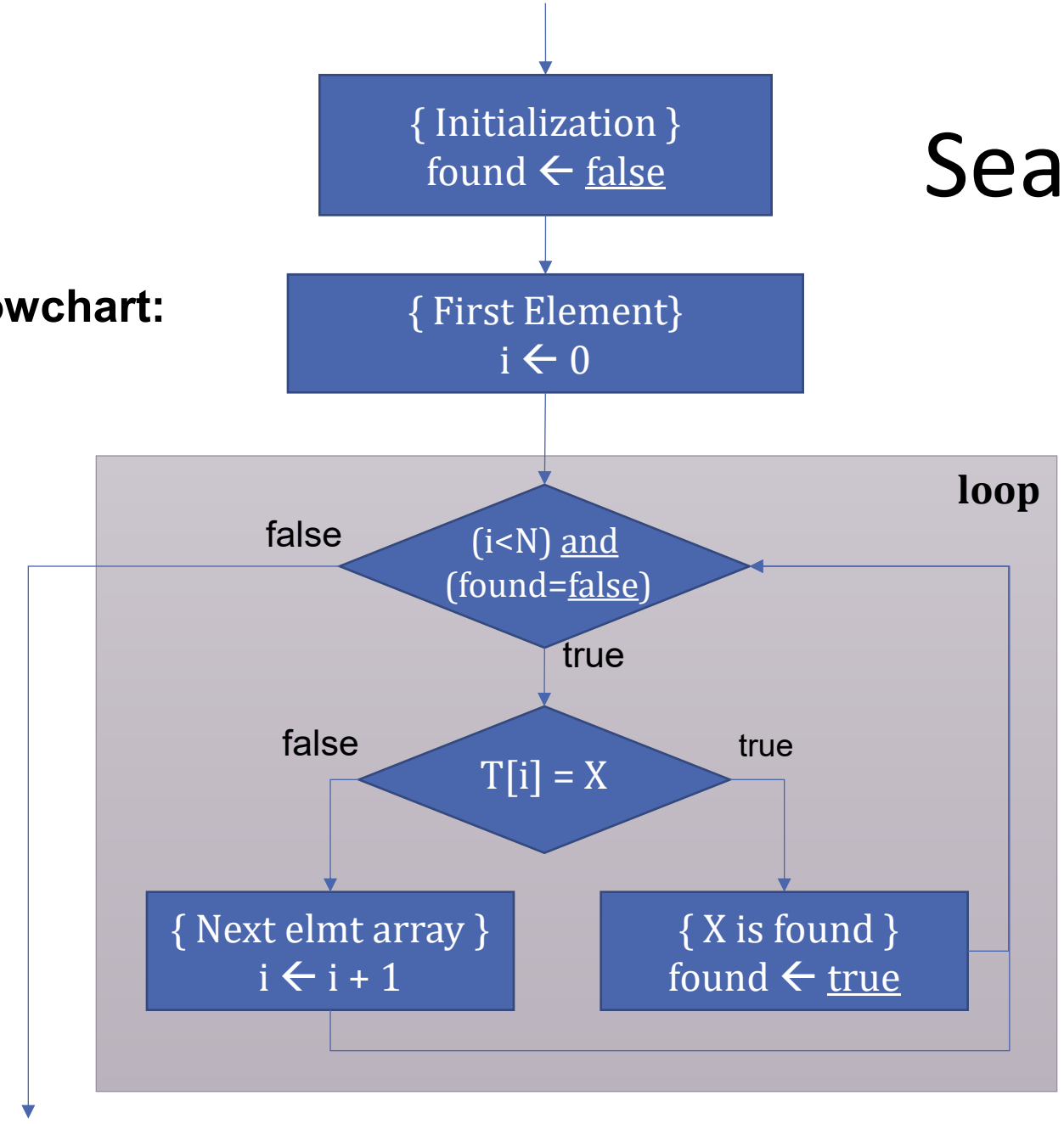
- Based on the previous example problem, create a program to find the **smallest** value among the elements of T.

Searching (1)
Looking up the index of the first appearance of a value

- **Searching** is an important process in array processing, as it is frequently performed on datasets stored within arrays.
- Given:
  - An array T of size N elements
  - A value X (of the same data type as the elements of T)
- Write a program to output the **first index in T where X is found**.
- Assumption: The array is not empty (minimum 1 element, N > 0).
- Examples:
  - N = 10; T contains: [9, 12, 30, -1, 0, 4, -1, 3, 30, 14]; X = -1; X is first found at index 3.
  - N = 8; T contains: [1, 3, 5, 8, -12, 90, 3, 5]; X = 0; X is not found in T.

Searching (2)

Flowchart:



Pseudocode:

```
...

i ← 0
found ← false
while (i<N) and (found=false) do
   if (T[i] = X) then
         found ← true
   else { T[i] ≠ X }
         i ← i + 1
{ i>=N or found=true }

...
```

```
# Program SearchArray
# Find the index where X is first found in T
# DECLARATIONS
# N : int; size of T
# T : array [0..N-1] of int
# i, X : int
# found : bool; determining whether X has been found or not

# ALGORITHM
# Assumption: the input array has been created; N is defined.

# Reading the value being sought, which is X
X = int(input())

# Searching from the first element
i = 0
found = False  # X is not yet found
while (i < N and found == False):
    if (T[i] == X):
        found = True   # X is found
    else:
        i = i + 1      # increment only when X is not found
# i = N or found = True

# Print result
if (found == True):  # X is found in T
    print (str(X) + " is found at index " + str(i))
else:  # found = false, X is not found
    print (str(X) + " is not found")
```

Searching (3)
Python Program

Exercise-4

- Based on the previous example, create a program to find the **last index** where X is found in T.
- Hint: the search process is performed "backwards," i.e., from the index of the last element to the first element.
  - What modifications must be made to the previous ALGORITHM?

Exercise-5

- A vector v = (v_0, v_1, v_2, v_3, v_4) is represented as an array of integers with 5 elements..
- Consider two vectors, each consisting of 5 elements, for example, V and U.
- Print the result of the addition of the two vectors.
- The addition of two vectors yields another vector, W, where the i-th element is: $W_i = V_i + U_i$

$$W = U + V = (v_0, v_1, v_2, v_3, v_4) + (u_0, u_1, u_2, u_3, u_4)$$

$$W = (v_0+u_0, v_1+u_1, v_2+u_2, v_3+u_3, v_4+u_4)$$